

Automation & QA Developer

Take-Home Assessment — Submission Documentation

Candidate	Candidate Name
Submission Date	25 May 2026
Role	Automation & QA Developer
Assessment Version	Take-Home Skills Assessment v1

This document provides a brief overview of all work submitted for the assessment. Full details are in the individual task files listed in the submission checklist below.

Submission Checklist

Task1_QA_Report_Candidate.pdf	Task 1 — QA bug table (5 issues) + root-cause analysis
Task2_Workflow_Candidate.json	Task 2 — n8n workflow export (Morning Brief)
Task2_README_Candidate.pdf	Task 2 — README: APIs, transformation, error path, credentials
Bonus_UptimeMonitor_Candidate.json	Bonus — n8n uptime monitor workflow export
Documentation_Candidate.pdf	This document — overall submission summary

Task 1 — Web App QA & Debug Report

App tested: RealWorld Conduit Demo — demo.realworld.io (Angular + Node/Express)

What I did:

- Exercised all major user flows: registration, login, article CRUD, comments, favourites, profile, logout.
- Applied boundary and negative-input testing (blank fields, 500+ char strings, special chars, emoji).
- Performed manual security probing: XSS payloads in text fields, browser storage inspection, unauthenticated API calls, HTTP header review.
- Ran accessibility audit: keyboard-only navigation, ARIA attribute inspection with axe DevTools.
- Tested API behaviour directly with HTTP requests (rate limiting, status codes, validation).

Issues found:

Critical	JWT stored in localStorage — XSS token theft risk
Critical	No rate limiting on login endpoint — brute-force possible
Critical	Stored XSS via Markdown article body (chosen for root-cause analysis)

High	Double-click on Publish creates duplicate articles
Medium	Pagination renders when all articles fit on one page
Medium	Missing ARIA labels on icon-only buttons (WCAG 2.1 violation)
Medium	Non-existent profile returns HTTP 200 with empty UI
Low	Tag field accepts arbitrary-length input, breaking tag cloud layout

Root-cause analysis was written for Issue 3 (Stored XSS). The vulnerability arises from the Angular frontend using `bypassSecurityTrustHtml()` to allow Markdown rendering, which simultaneously disables Angular's XSS guard. The marked parser also lacks a sanitisation step. Fix requires server-side `sanitize-html` + client-side `DOMPurify` + a strict `Content-Security-Policy` header.

Task 2 — n8n API Integration Workflow

Workflow name: **Morning Brief – GitHub AI Repos**. The workflow runs every hour, fetches trending AI repositories from GitHub, enriches the top 5, applies a star-count threshold, and posts a formatted digest to Discord.

Key design decisions:

- GitHub REST API chosen for richness, reliability, and generous rate limits (5 000 req/h authenticated).
- Star threshold of 1 000: cleanly separates established projects from new/niche ones in the AI space.
- Two enrichment paths (hot/regular) keep the workflow extensible — different data is fetched per tier.
- `continueOnFail=true` on all HTTP nodes prevents silent crashes from transient API failures.
- Error Trigger + Discord alert ensures failures are always surfaced to the team within seconds.
- No secrets hardcoded: GitHub token and Discord webhook URL stored in n8n Credential store.

Bonus — Uptime Monitor

Workflow name: **Uptime Monitor – demo.realworld.io**. Pings the app every 5 minutes, evaluates HTTP status and response time, retries up to 3 times on failure, alerts Discord on non-200 responses or responses slower than 3 seconds, and persists rolling statistics for a daily 08:00 summary report.

Bonus features implemented:

- 3-attempt retry logic with 15-second wait between attempts (using the n8n Wait node).
- Response-time tracking: checks if GET response exceeds 3 000 ms and flags as SLOW.
- `staticData` persistence: rolling 288-check window (24 h at 5-min interval) stored in workflow static data.
- Daily summary: separate Schedule trigger fires at 08:00, reads stored stats, calculates uptime % and average response time, posts to Discord.
- Two-level error handling: `continueOnFail` on HTTP nodes + Error Trigger for workflow-level failures.